Le Back-Office

Symfony étant ce qu'il est (extrêmement puissant et pensé pour des applications métier complexes), on ne va pas lancer un projet Symfony pour n'afficher qu'une partie **publique**.

Même pour un site vitrine « simple », l'idée ici est de mettre en place que l'on va appeler un **back office**, c'est-à-dire une partie « administration » du site, qui ne sera accessible qu'aux propriétéiares du site et leur permettra de gérer les contenus affichés dans la partie publique du site.

Nous allons donc nous servir du projet vu dans le cours précédent pour ajouter des CRUDs dans le back-office.

Table des matières

1. Générer les CRUD 2
2. Analyser les routes crées 2

1. Générer les CRUD

Vous n'allez pas le croire, mais si : on peut également générer les CRUDs avec le **MakerBundle**, encore lui.

Dans notre projet vitrine, nous allons lancer ces deux commandes :

```
php bin/console make:crud Produit
```

```
php bin/console make:crud Categorie
```

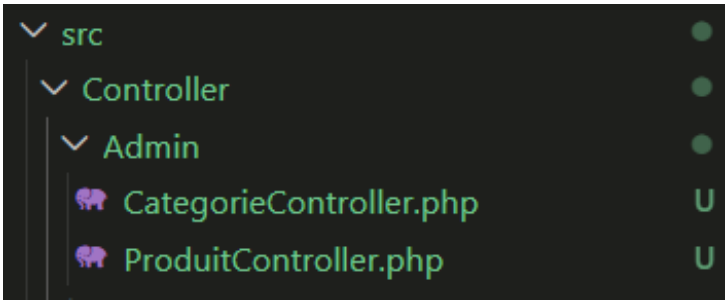
Et pour chacune d'entre elle, lorsque qu'il va nous demander quel doit être le nom du Controller généré, nous allons les **préfixer**. On va vouloir que tout soit bien rangé, et surtout on va séparer les controllers de l'admin des controllers **publics**.

On va donc lui dire de nommer le controller **Admin\ProduitController**, puis **Admin\CategorieController**, comme ceci :

```
nicolas@MUSS:~/wsldev/projects/vitrine$ php bin/console make:crud Produit

 Choose a name for your controller class (e.g. ProduitController) [ProduitController]:
 > Admin\ProduitController
```

Le MakerBundle va alors nous créer, pour chaque entité, 5 routes, 5 templates.

On se retrouve avec cette structure de fichiers :



2. Analyser les routes crées

! Étant donné que les routes crées par cette génération sont les mêmes pour nos deux entités, nous allons nous attarder uniquement sur les produits. !

Nous avons donc 5 routes qui sont créées dans chaque controller, et chacune d'entre elle correspond à un template également généré. Voici les 5 routes en question :

- index
- new
- edit
- show
- delete

Afin de pouvoir y accéder plus simplement, nous allons créer une interface admin très rapidement afin de pouvoir naviguer entre les différentes vues. Pour cela nous allons créer un template **base.html.twig** dans le dossier admin : il sera notre **layout** principal pour toute l'administration

```
{# templates/admin/base.html.twig #}
{% extends 'base.html.twig' %}

{% block body %}
<div class="container-fluid">
    <div class="row">

        {# Sidebar #}
        <nav class="col-md-2 bg-dark min-vh-100 p-3">
            <h5 class="text-white mb-4">Administration</h5>
            <ul class="nav flex-column">
                <li class="nav-item">
                    <a class="nav-link text-white" href="{{ path('app_admin_dashboard') }}">
                        Dashboard
                    </a>
                </li>
                <li class="nav-item">
                    <a class="nav-link text-white" href="{{ path('app_admin_produit_index') }}">
                        Produits
                    </a>
                </li>
                <li class="nav-item">
                    <a class="nav-link text-white" href="{{ path('app_admin_categorie_index') }}">
                        Catégories
                    </a>
                </li>
                <li class="nav-item mt-4">
                    <a class="nav-link text-danger" href="{{ path('app_logout') }}">
                        Déconnexion
                    </a>
                </li>
            </ul>
        </nav>

        {# Contenu principal #}
        <main class="col-md-10 p-4">
            {% block admin_content %}{% endblock %}
        </main>

    </div>
</div>
{% endblock %}
```

Puis ensuite nous allons adapter nos templates de route un par un. Voici un exemple avec la route /admin/produit/index. Deux actions à effectuer ici :

- On dit que notre template hérite de **admin/base.html.twig** au lieu de **base.html.twig**
- On change le block **body** pour le remplacer par le block **admin_content**

Enfin, on met à jour notre **dashboard** histoire d'avoir des liens de menu fonctionnels lorsqu'on se connecte :

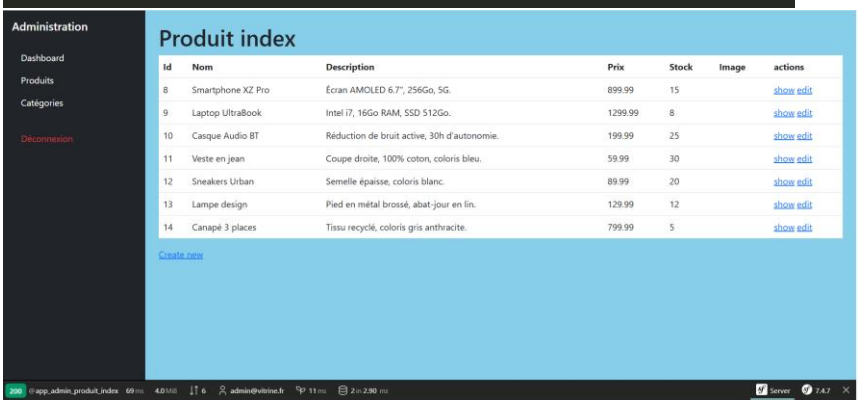
```
{# templates/admin/dashboard.html.twig #}
{% extends 'admin/base.html.twig' %}

{% block title %}Dashboard{% endblock %}

{% block admin_content %}
    <h1>Bienvenue {{ app.user.nom }} 👋</h1>
    <p>Vous êtes connecté en tant qu'administrateur.</p>
{% endblock %}
```

On va donc arriver avec un Dashboard comme ceci :

On va pouvoir détailler les routes maintenant :

- **index** : *(Le **R** dans CRUD)* c'est cette route qui va être le point de départ de notre CRUD. Elle consiste en une **LISTE** de tous les produits que nous avons, c'est pour ça que nous avons un **findAll()** dans la méthode qui accompagne la Route dans le controller :

```
#[Route(name: 'app_admin_produit_index', methods: ['GET'])]
public function index(ProduitRepository $produitRepository): Response
{
    return $this->render('admin/produit/index.html.twig', [
        'produits' => $produitRepository->findAll(),
    ]);
}
```



- **new** : *(Le **C** dans CRUD)* C'est la route à laquelle on accède quand on clique sur le bouton généré en bas de la liste « Create new ». Elle contient un formulaire permettant d'ajouter un nouveau produit à la liste. C'est pour ça qu'elle contient une création de Formulaire ainsi qu'une **persistance** du nouvel objet crée dans la méthode qui décrit la route :

```
#[Route('/new', name: 'app_admin_produit_new', methods: ['GET', 'POST'])]
public function new(Request $request, EntityManagerInterface $entityManager): Response
{
    $produit = new Produit();
    $form = $this->createForm(ProduitType::class, $produit);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->persist($produit);
        $entityManager->flush();

        return $this->redirectToRoute('app_admin_produit_index', [], Response::HTTP_SEE_OTHER);
    }

    return $this->render('admin/produit/new.html.twig', [
        'produit' => $produit,
        'form' => $form,
    ]);
}
```

- **edit** : *(Le **U** dans CRUD)* C'est la route qui va nous permettre de **modifier** un produit. On y accède en cliquant sur le lien « edit » dans la colonne **actions** qui a été générée. On y retrouve là aussi un formulaire et la **persistance** en base des modifications. En plus de ça, on y retrouve le paramètre **{id}** qui est utilisé pour retrouver **$produit** sans appeler le Repository : En effet, vu qu'ici on est sur un CRUD généré, Symfony utilise ce que l'on appelle le **ParamConverter** pour charger le produit

```
#[Route('/{id}/edit', name: 'app_admin_produit_edit', methods: ['GET', 'POST'])]
public function edit(Request $request, Produit $produit, EntityManagerInterface $entityManager): Response
{
    $form = $this->createForm(ProduitType::class, $produit);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->flush();

        return $this->redirectToRoute('app_admin_produit_index', [], Response::HTTP_SEE_OTHER);
    }

    return $this->render('admin/produit/edit.html.twig', [
        'produit' => $produit,
        'form' => $form,
    ]);
}
```

- **show** : *(Le **R** dans CRUD, encore)* C'est la route qui va nous permettre de consulter les informations du produit qui n'apparaissent pas dans la vue **index**. Dans la route, on retrouve le **ParamConverter** qui nous permet de passer le produit directement dans **$produit** sans utiliser le Repository :

```
#[Route('/{id}', name: 'app_admin_produit_show', methods: ['GET'])]
public function show(Produit $produit): Response
{
    return $this->render('admin/produit/show.html.twig', [
        'produit' => $produit,
    ]);
}
```

- **delete** : *(Le **D** dans CRUD)* Ici on gère tout simplement une suppression. Dans la route en retrouve l'**entityManager** sur lequel on appelle la méthode **remove** afin de supprimer l'objet passé en paramètre, puis on est redirigé vers la vue **index**

```
#[Route('/{id}', name: 'app_admin_produit_delete', methods: ['POST'])]
public function delete(Request $request, Produit $produit, EntityManagerInterface $entityManager):
Response
{
 if ($this->isCsrfTokenValid('delete'.$produit->getId(), $request->getPayload()->getString('_token')))
{
    $entityManager->remove($produit);
    $entityManager->flush();
  }

  return $this->redirectToRoute('app_admin_produit_index', [], Response::HTTP_SEE_OTHER);
}
```

Et voilà on a fait le tour ☺

EXERCICE

Je vous laisse faire la même chose pour les catégories, et, comme à la fin de chaque cours ou presque : Stylisez moi tout ça ! Mettez moi du Bootstrap, rendez-moi cette administration agréable à naviguer. Tout le cœur du code est là, il ne reste plus qu'à y mettre un coup de vernis.

Si je vous fais pratiquer autant le design sur Symfony, ce n'est pas pour le plaisir, c'est pour que vous vous appropriez la structure du projet. Symfony est un framework **COMPLEXE**, qui implique la création de beaucoup de fichiers. Il faut que vous soyez à l'aise pour aller modifier des trucs à droite à gauche : Le label d'un bouton, la largeur d'une colonne, remplacer l'id d'une catégorie par son nom dans un select…

À vous de jouer ☺